

Week 4: Data Structures: Lists & Tuples

Learning Outcome/Trait Assessment:

- Store and manipulate collections of data using lists.
- Understand list methods for adding, removing, and organizing elements.
- Differentiate between lists and tuples based on mutability.
- Select the appropriate data structure (list or tuple) for a given scenario.
- **Trait Assessment:** Effective data organization, appropriate data structure selection, logical application of list/tuple operations, attention to mutability/immutability.

Detailed Lecture Topics:

1. Introduction to Data Structures:

- Why do we need data structures? (Beyond single variables, organizing related data).
- Concept of collections.

2. Lists (list): The Mutable, Ordered Sequence

- **Definition:** Ordered, changeable (mutable), allows duplicate members. Defined by square brackets [].
- **Creating Lists:**
 - Empty list: `my_list = []`
 - With elements: `numbers = [1, 2, 3]`, `fruits = ["apple", "banana", "cherry"]`
 - Using list() constructor: `my_list = list(("a", "b", "c"))`
- **Accessing List Elements:**
 - **Indexing:** `list[index]` (positive indexing from 0, negative indexing from -1).
 - **Slicing:** `list[start:end:step]` (extracting sub-lists).
- **Modifying Lists (Mutability):**
 - Changing elements: `fruits[0] = "orange"`
 - **Adding Elements:**
 - `append(item)`: Adds to the end.
 - `insert(index, item)`: Inserts at a specific index.
 - `extend(another_list)`: Appends elements from another iterable.
 - **Removing Elements:**
 - `remove(item)`: Removes the first occurrence of a specific value.
 - `pop(index)`: Removes and returns the element at a specific index (or last if no index).
 - `del list[index]` or `del list`: Deletes elements or the entire list.
 - `clear()`: Empties the list.
- **Other Useful List Methods & Functions:**
 - `len(list)`: Returns number of items.
 - `sort()`: Sorts the list in-place. `sorted()`: Returns a new sorted list.
 - `reverse()`: Reverses the order of elements in-place.

- `count(item)`: Returns number of occurrences of an item.
 - `index(item)`: Returns the index of the first occurrence of an item.
 - **Iterating through Lists:** (Revisit for `item in list:` and `for i in range(len(list)):`)
 - List Comprehensions (brief introduction as a concise way to create lists, for advanced students).
3. **Tuples (tuple): The Immutable, Ordered Sequence**
- **Definition:** Ordered, unchangeable (immutable), allows duplicate members. Defined by parentheses `()`.
 - **Creating Tuples:**
 - Empty tuple: `my_tuple = ()`
 - With elements: `coordinates = (10, 20)`, `colors = ("red", "green", "blue")`
 - Single item tuple (crucial syntax): `single_item = ("apple",)`
 - Without parentheses (tuple packing): `x, y, z = 1, 2, 3`
 - **Accessing Tuple Elements:**
 - Indexing and slicing (same as lists).
 - **Immutability Demonstration:** Show attempts to modify, add, or remove elements resulting in `TypeError`.
 - **When to Use Tuples:**
 - When data should not change (e.g., coordinates, database records, function arguments).
 - As dictionary keys (because they are immutable).
 - More memory efficient than lists for large, fixed collections.
4. **Converting Between Lists and Tuples:**
- `list(tuple_name)`: Converts a tuple to a list.
 - `tuple(list_name)`: Converts a list to a tuple.

Detailed Labs:

- **Lab 4.1: Student Management System (List Operations)**
 - **Objective:** Students will build a simple program to manage a list of student names, demonstrating various list operations.
 - **Tasks (Common for Local & Colab):**
 1. Create a new Python file (`lab4_1.py`) or Colab code cell.
 2. Initialize an empty list called `student_names`.
 3. **Add Students:** Prompt the user to enter 3-5 student names and `append()` each name to the list. Print the current list.
 4. **Find a Student:** Ask the user for a name to search for. Use `if item in list:` to check if the student is present. If found, print their index using `index()`.
 5. **Remove a Student:** Ask the user for a student name to remove. Use `remove()` to delete the student. Handle the case where the student is not found (e.g., using `if item in list:` before `remove()`). Print the updated list.

6. **Sort Students:** Sort the `student_names` list alphabetically using `sort()`. Print the sorted list.
 7. **Count Students:** Print the total number of students in the list using `len()`.
 8. **Challenge:** Implement a simple menu-driven system using a while loop (from Week 3) where the user can choose to add, remove, search, sort, or exit.
- **Lab 4.2: Geometric Calculations (Tuples & Immutability)**
 - **Objective:** Students will use tuples to represent 2D points and calculate the distance between them, highlighting tuple immutability.
 - **Tasks (Common for Local & Colab):**
 1. Create a new Python file (`lab4_2.py`) or Colab code cell.
 2. **Define Points:** Define two points as tuples: `point1 = (3, 4)` and `point2 = (6, 8)`.
 3. **Access Coordinates:** Access and print the x and y coordinates of both points.
 - *Example:* "Point 1: x = 3, y = 4"
 4. **Attempt Modification (Illustrate Immutability):** Try to change `point1`'s x-coordinate (e.g., `point1[0] = 5`). Observe and explain the `TypeError`. (Comment out this line after demonstrating).
 5. **Calculate Distance:**
 - Import the math module: `import math`.
 - Use the distance formula: $(x_2 - x_1)^2 + (y_2 - y_1)^2$.
 - Calculate the distance between `point1` and `point2` using `math.sqrt()` and `**` (power operator).
 - Print the calculated distance.
 6. **Challenge:** Prompt the user to enter coordinates for two new points and then calculate the distance between them. Ensure type conversion from `input()`.

Detailed Activities:

- **Activity 4.1: List vs. Tuple: Scenario Matching**
 - **Format:** Small group discussion, followed by a class debrief.
 - **Instructions:** Present students with a list of real-world data scenarios. For each scenario, they must decide whether a list or a tuple would be the more appropriate Python data structure and explain their reasoning (focus on mutability/immutability and ordered nature).
 - **Scenarios:**
 1. A collection of daily temperatures recorded over a week.
 2. The RGB color values for a specific shade (e.g., (255, 0, 0) for red).
 3. Items in a shopping cart that can be added or removed.
 4. The days of the week in order.
 5. Student scores on a series of assignments, which might be updated.
 6. GPS coordinates for a fixed landmark.
 - **Purpose:** Reinforce understanding of when to use which data structure based on their

characteristics.

- **Activity 4.2: Interactive To-Do List (using List Operations)**

- **Format:** Individual coding challenge.
- **Instructions:** Students will implement a very basic text-based to-do list manager using what they've learned about lists.
- **Features to include (minimum):**
 - An empty list `todo_list` initially.
 - A while loop that presents a menu:
 1. Add a task
 2. View tasks
 3. Mark task as complete (remove by index or name)
 4. Exit
 - Use `input()` to get user choices.
 - Use list methods (`append()`, `remove()` or `pop()`, `len()`) to manage the `todo_list`.
- **Purpose:** Practical application of list operations in a mini-project context, integrating loops and conditionals.

Sample Snippets for Class and Colab Ready

1. Lists: Creation, Access, Modification

VS Code (`list_operations.py`):

Python

```
# list_operations.py
```

```
print("--- List Creation and Access ---")
empty_list = []
print(f"Empty list: {empty_list}")
```

```
fruits = ["apple", "banana", "cherry", "date"]
print(f"Original fruits list: {fruits}")
```

```
# Accessing elements by index
print(f"First fruit: {fruits[0]}") # apple
print(f"Last fruit (negative index): {fruits[-1]}") # date
```

```

# Slicing
print(f"Fruits from index 1 to 2 (exclusive): {fruits[1:3]}") # ['banana', 'cherry']
print(f"All fruits except first: {fruits[1:]}") # ['banana', 'cherry', 'date']
print(f"Last two fruits: {fruits[-2:]}") # ['cherry', 'date']
print(f"Every second fruit: {fruits[::2]}") # ['apple', 'cherry']

print("\n--- Modifying List Elements ---")
fruits[1] = "blueberry" # Change an element
print(f"After changing index 1: {fruits}")

print("\n--- Adding Elements to List ---")
fruits.append("elderberry") # Add to the end
print(f"After append: {fruits}")

fruits.insert(1, "grape") # Insert at specific index
print(f"After insert at index 1: {fruits}")

more_fruits = ["kiwi", "lemon"]
fruits.extend(more_fruits) # Add elements from another list
print(f"After extend: {fruits}")

print("\n--- Removing Elements from List ---")
fruits.remove("cherry") # Remove by value (first occurrence)
print(f"After remove 'cherry': {fruits}")

popped_fruit = fruits.pop(2) # Remove by index (and get the item)
print(f"Popped fruit at index 2: {popped_fruit}")
print(f"After pop at index 2: {fruits}")

del fruits[0] # Delete by index
print(f"After del index 0: {fruits}")

# fruits.clear() # Empties the list
# print(f"After clear: {fruits}")

print("\n--- Other List Operations ---")
numbers = [3, 1, 4, 1, 5, 9, 2]
print(f"Original numbers: {numbers}")

print(f"Length of numbers list: {len(numbers)}")
print(f"Count of 1 in numbers: {numbers.count(1)}")

```

```
print(f"Index of first '4': {numbers.index(4)}")
```

```
numbers.sort() # Sorts in-place (modifies the original list)
```

```
print(f"Sorted numbers (in-place): {numbers}")
```

```
numbers.reverse() # Reverses in-place
```

```
print(f"Reversed numbers (in-place): {numbers}")
```

```
# Using sorted() to get a new sorted list without changing original
```

```
unsorted_numbers = [5, 2, 8, 1, 9]
```

```
new_sorted_numbers = sorted(unsorted_numbers)
```

```
print(f"Unsorted numbers: {unsorted_numbers}")
```

```
print(f"New sorted list: {new_sorted_numbers}")
```

```
print("\n--- Iterating through a List ---")
```

```
for fruit in fruits:
```

```
    print(f"I like {fruit}")
```

```
for i in range(len(fruits)):
```

```
    print(f"Fruit at index {i}: {fruits[i]}")
```

Google Colab (in a code cell):

Python

```
# list_operations in Colab
```

```
print("--- List Creation and Access ---")
```

```
empty_list = []
```

```
print(f"Empty list: {empty_list}")
```

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
print(f"Original fruits list: {fruits}")
```

```
# Accessing elements by index
```

```
print(f"First fruit: {fruits[0]}")
```

```
print(f"Last fruit (negative index): {fruits[-1]}")
```

```

# Slicing
print(f"Fruits from index 1 to 2 (exclusive): {fruits[1:3]}")
print(f"All fruits except first: {fruits[1:]}")
print(f>Last two fruits: {fruits[-2:]}")
print(f"Every second fruit: {fruits[::2]}")

print("\n--- Modifying List Elements ---")
fruits[1] = "blueberry" # Change an element
print(f"After changing index 1: {fruits}")

print("\n--- Adding Elements to List ---")
fruits.append("elderberry") # Add to the end
print(f"After append: {fruits}")

fruits.insert(1, "grape") # Insert at specific index
print(f"After insert at index 1: {fruits}")

more_fruits = ["kiwi", "lemon"]
fruits.extend(more_fruits) # Add elements from another list
print(f"After extend: {fruits}")

print("\n--- Removing Elements from List ---")
fruits.remove("cherry") # Remove by value (first occurrence)
print(f"After remove 'cherry': {fruits}")

popped_fruit = fruits.pop(2) # Remove by index (and get the item)
print(f"Popped fruit at index 2: {popped_fruit}")
print(f"After pop at index 2: {fruits}")

del fruits[0] # Delete by index
print(f"After del index 0: {fruits}")

# fruits.clear() # Empties the list
# print(f"After clear: {fruits}")

print("\n--- Other List Operations ---")
numbers = [3, 1, 4, 1, 5, 9, 2]
print(f"Original numbers: {numbers}")

print(f"Length of numbers list: {len(numbers)}")
print(f"Count of 1 in numbers: {numbers.count(1)}")

```

```

print(f"Index of first '4': {numbers.index(4)}")

numbers.sort() # Sorts in-place (modifies the original list)
print(f"Sorted numbers (in-place): {numbers}")

numbers.reverse() # Reverses in-place
print(f"Reversed numbers (in-place): {numbers}")

# Using sorted() to get a new sorted list without changing original
unsorted_numbers = [5, 2, 8, 1, 9]
new_sorted_numbers = sorted(unsorted_numbers)
print(f"Unsorted numbers: {unsorted_numbers}")
print(f"New sorted list: {new_sorted_numbers}")

print("\n--- Iterating through a List ---")
for fruit in fruits:
    print(f"I like {fruit}")

for i in range(len(fruits)):
    print(f"Fruit at index {i}: {fruits[i]}")

```

2. Tuples: Creation, Access, Immutability

VS Code (tuple_operations.py):

Python

```

# tuple_operations.py

print("--- Tuple Creation and Access ---")
empty_tuple = ()
print(f"Empty tuple: {empty_tuple}")

colors = ("red", "green", "blue", "yellow")
print(f"Original colors tuple: {colors}")

# Single item tuple requires a comma
single_item_tuple = ("single",)

```

```
print(f"Single item tuple: {single_item_tuple}, Type: {type(single_item_tuple)}")
not_a_tuple = ("single") # This is just a string in parentheses
print(f"Not a tuple: {not_a_tuple}, Type: {type(not_a_tuple)}")
```

```
# Accessing elements by index
print(f"First color: {colors[0]}") # red
print(f"Last color (negative index): {colors[-1]}") # yellow
```

```
# Slicing
print(f"Colors from index 1 to 2 (exclusive): {colors[1:3]}") # ('green', 'blue')
```

```
print("\n--- Demonstrating Tuple Immutability ---")
try:
    colors[0] = "orange" # Attempt to change an element
except TypeError as e:
    print(f"Error trying to modify tuple: {e}")
```

```
try:
    colors.append("purple") # Attempt to add an element
except AttributeError as e:
    print(f"Error trying to append to tuple: {e}")
```

```
try:
    del colors[0] # Attempt to delete an element
except TypeError as e:
    print(f"Error trying to delete from tuple: {e}")
```

```
print(f"Tuple remains unchanged: {colors}") # The tuple is still the same
```

```
print("\n--- Other Tuple Operations ---")
point = (10, 20)
x_coord, y_coord = point # Tuple unpacking
print(f"x_coord: {x_coord}, y_coord: {y_coord}")
```

```
print(f"Count of 'blue': {colors.count('blue')}")
print(f"Index of 'yellow': {colors.index('yellow')}")
```

```
print("\n--- Converting between List and Tuple ---")
my_list = [1, 2, 3, 4]
my_tuple_from_list = tuple(my_list)
```

```
print(f"List converted to tuple: {my_tuple_from_list}, Type: {type(my_tuple_from_list)}")
```

```
my_tuple = ("alpha", "beta", "gamma")
```

```
my_list_from_tuple = list(my_tuple)
```

```
print(f"Tuple converted to list: {my_list_from_tuple}, Type: {type(my_list_from_tuple)}")
```

Google Colab (in a code cell):

Python

```
# tuple_operations in Colab
```

```
print("--- Tuple Creation and Access ---")
```

```
empty_tuple = ()
```

```
print(f"Empty tuple: {empty_tuple}")
```

```
colors = ("red", "green", "blue", "yellow")
```

```
print(f"Original colors tuple: {colors}")
```

```
# Single item tuple requires a comma
```

```
single_item_tuple = ("single",)
```

```
print(f"Single item tuple: {single_item_tuple}, Type: {type(single_item_tuple)}")
```

```
not_a_tuple = ("single") # This is just a string in parentheses
```

```
print(f"Not a tuple: {not_a_tuple}, Type: {type(not_a_tuple)}")
```

```
# Accessing elements by index
```

```
print(f"First color: {colors[0]}")
```

```
print(f"Last color (negative index): {colors[-1]}")
```

```
# Slicing
```

```
print(f"Colors from index 1 to 2 (exclusive): {colors[1:3]}")
```

```
print("\n--- Demonstrating Tuple Immutability ---")
```

```
try:
```

```
    colors[0] = "orange" # Attempt to change an element
```

```
except TypeError as e:
```

```
    print(f"Error trying to modify tuple: {e}")
```

```

try:
    colors.append("purple") # Attempt to add an element
except AttributeError as e:
    print(f"Error trying to append to tuple: {e}")

try:
    del colors[0] # Attempt to delete an element
except TypeError as e:
    print(f"Error trying to delete from tuple: {e}")

print(f"Tuple remains unchanged: {colors}")

print("\n--- Other Tuple Operations ---")
point = (10, 20)
x_coord, y_coord = point # Tuple unpacking
print(f"x_coord: {x_coord}, y_coord: {y_coord}")

print(f"Count of 'blue': {colors.count('blue')}")
print(f"Index of 'yellow': {colors.index('yellow')}")

print("\n--- Converting between List and Tuple ---")
my_list = [1, 2, 3, 4]
my_tuple_from_list = tuple(my_list)
print(f"List converted to tuple: {my_tuple_from_list}, Type: {type(my_tuple_from_list)}")

my_tuple = ("alpha", "beta", "gamma")
my_list_from_tuple = list(my_tuple)
print(f"Tuple converted to list: {my_list_from_tuple}, Type: {type(my_list_from_tuple)}")

```